

Distributed Systems

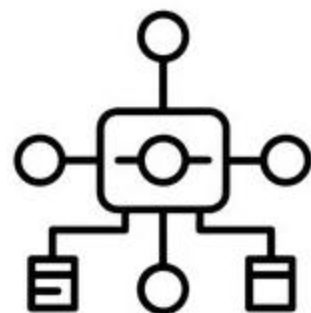
Distributed Systems/Platforms



Universidad de
La Sabana

FACULTAD DE INGENIERÍA

Agenda



Distributed Systems

Why

- You can buy...
- Fallacies of distributed computing
- QA
 - Availability
 - Scalability
 - Performance
 - Reliability

Styles

- Big Computing
- Big Data
- Space-Based

Patterns

- Sharding
 - Definition
 - Design keys
- CQRS

Theorems

- CAP & PACELC
- Databases mapped

Best Practices

- DynamoDB
- MongoDB

What Extend

- Hadoop/EMR
- Distributed Databases

You can buy hardware

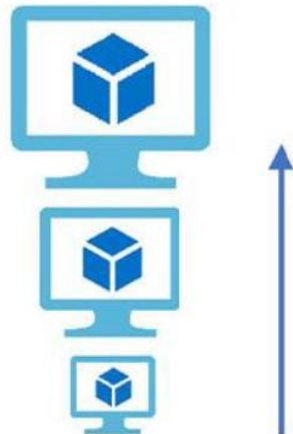
A bank buy/rent hardware capacity for his ESB until hard-limit, and its excuse is success based on number of users then...

- Stress load and breaker circuit → Hard response
- Apology: Take a technical decision completely based on business needs → Reputational damage
- Comparative advantage: His competitors on tech side can deal with that ← Simple mastery

I am not sure if this is solved, followed The Innovators Dylemma (Harvard, 1997) and try using “El banco del mañana”



Hard Limit



Decoupled or Exposed services: LB, MQ, API GW



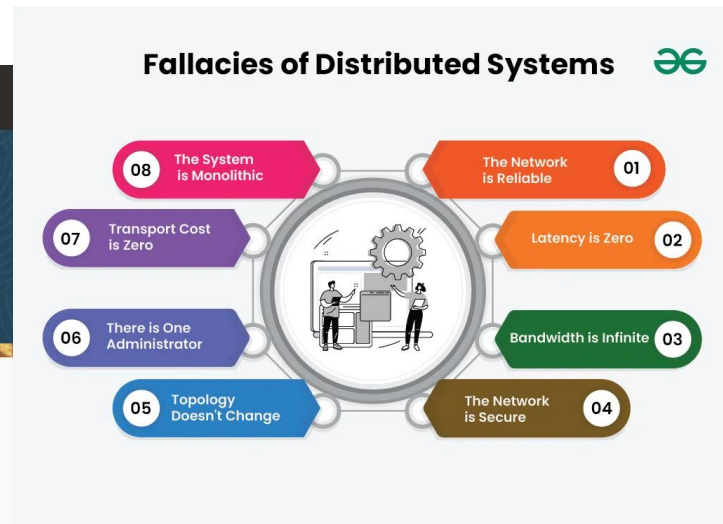
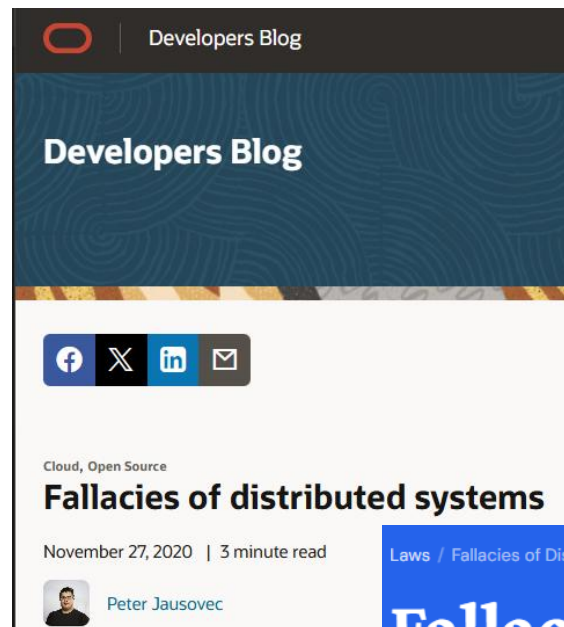
Is that the solution:
Pros and Cons ?

Orchestrator,
ASG,
Containers



Fallacies of Distributed Computing

	The network is reliable
	Latency is zero
	Bandwidth is infinite
	The network is secure
	Topology doesn't change
	There is one administrator
	Transport cost is zero
	The network is homogeneous



Don't lie yourself ! Here is the real life on the cloud!

(..), The System is Monolithic, The System is Fully Observable, The System is Always On, There is One Root Cause, Failures are Rare **[GoG]**.

[DDiA] add consequences of it: Fault and Partial Errors, Unreliable Networks, Unreliable Clocks and Others.

Why ?

1. Reliable, Scalable, and Maintainable Applications...

Thinking About Data Systems

Reliability

Hardware Faults
Software Errors
Human Errors

How Important Is Reliability?

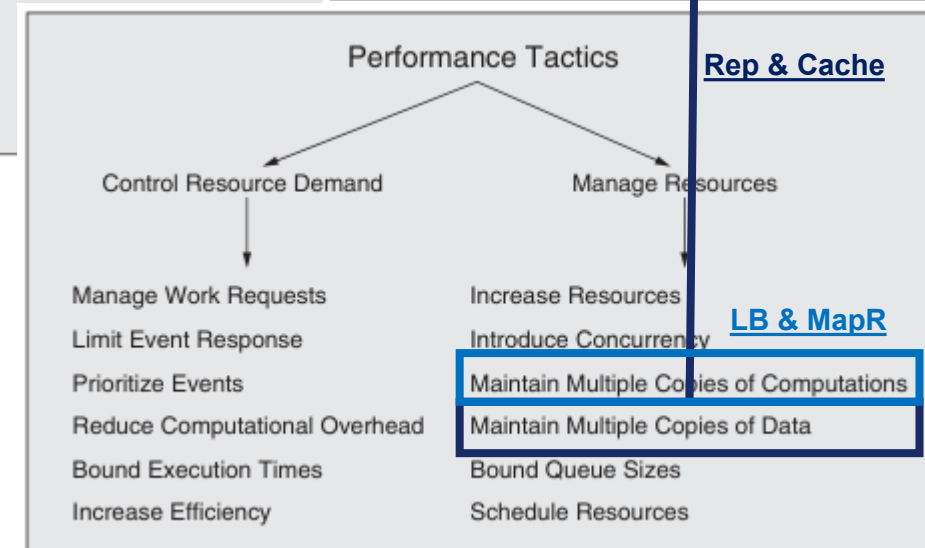
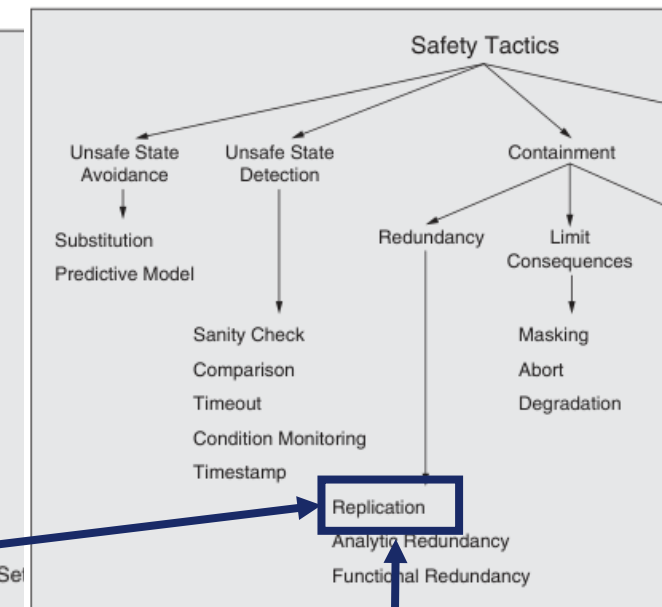
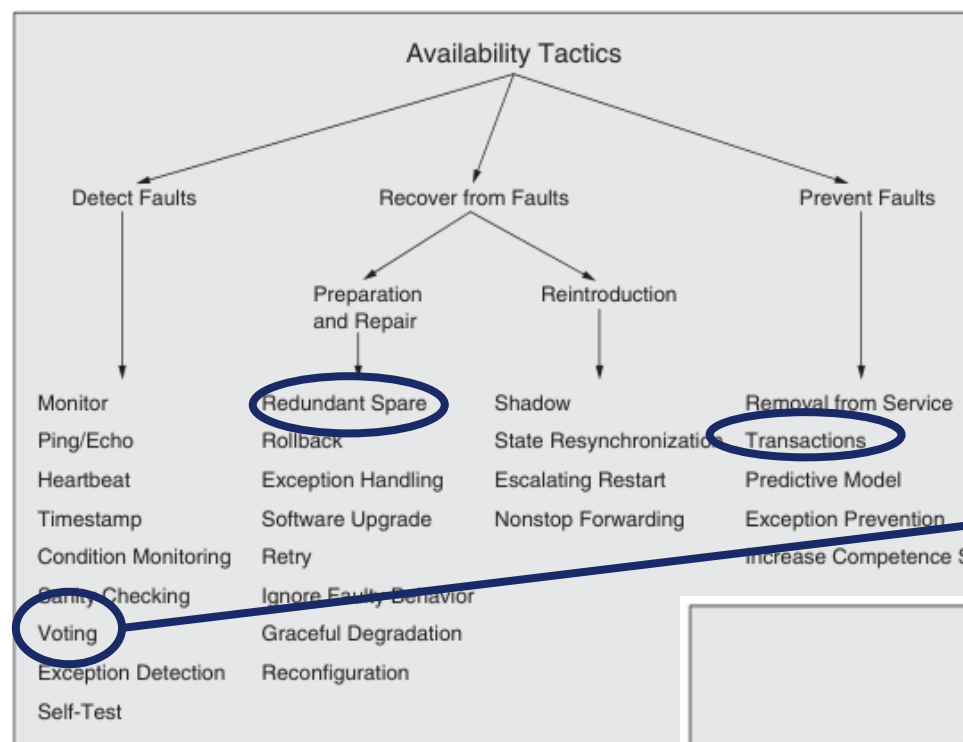
Scalability

Describing Load
Describing Performance
Approaches for Coping with Load

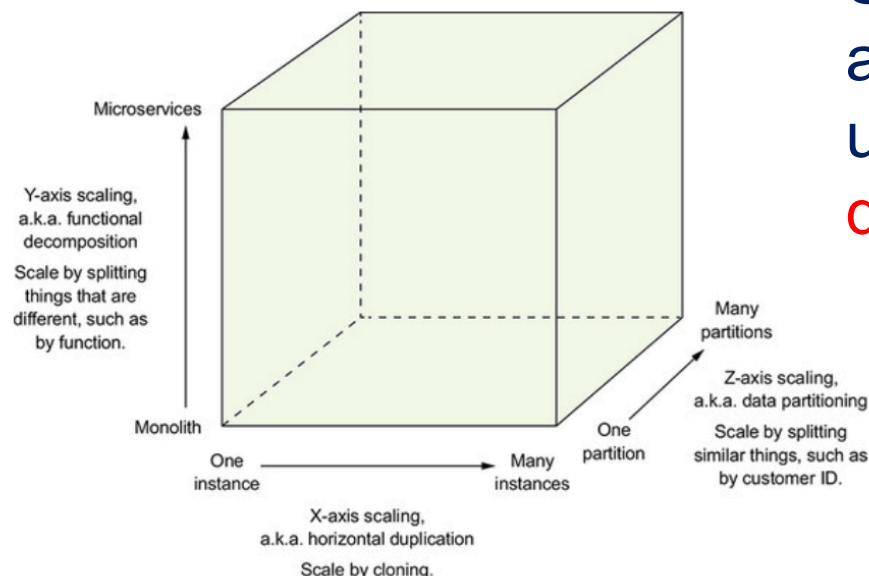
Maintainability

Operability: Making Life Easy for Operations
Simplicity: Managing Complexity
Evolvability: Making Change Easy

Summary



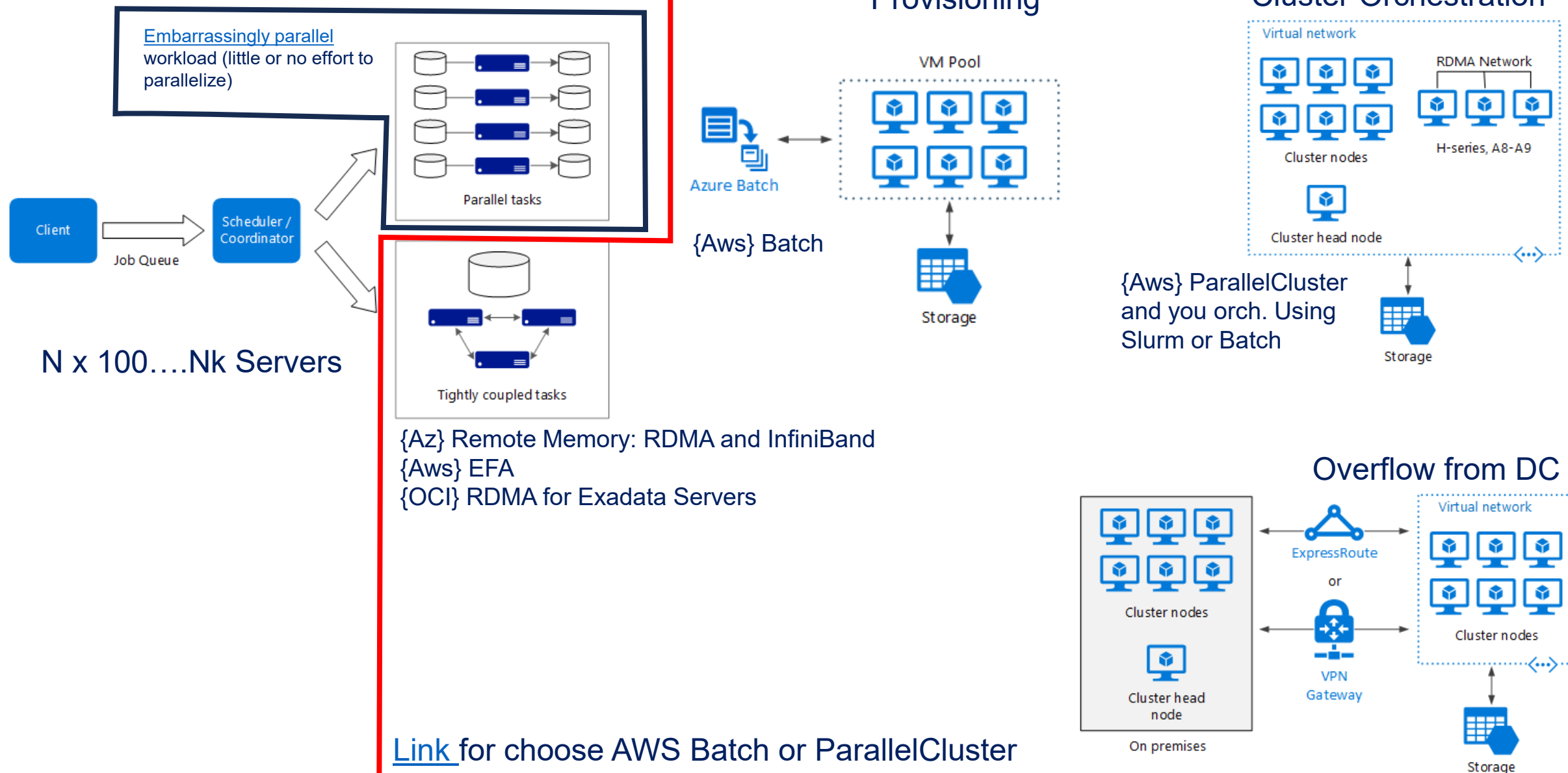
SAiP didn't notice
about Scalability
using the partition
dimension



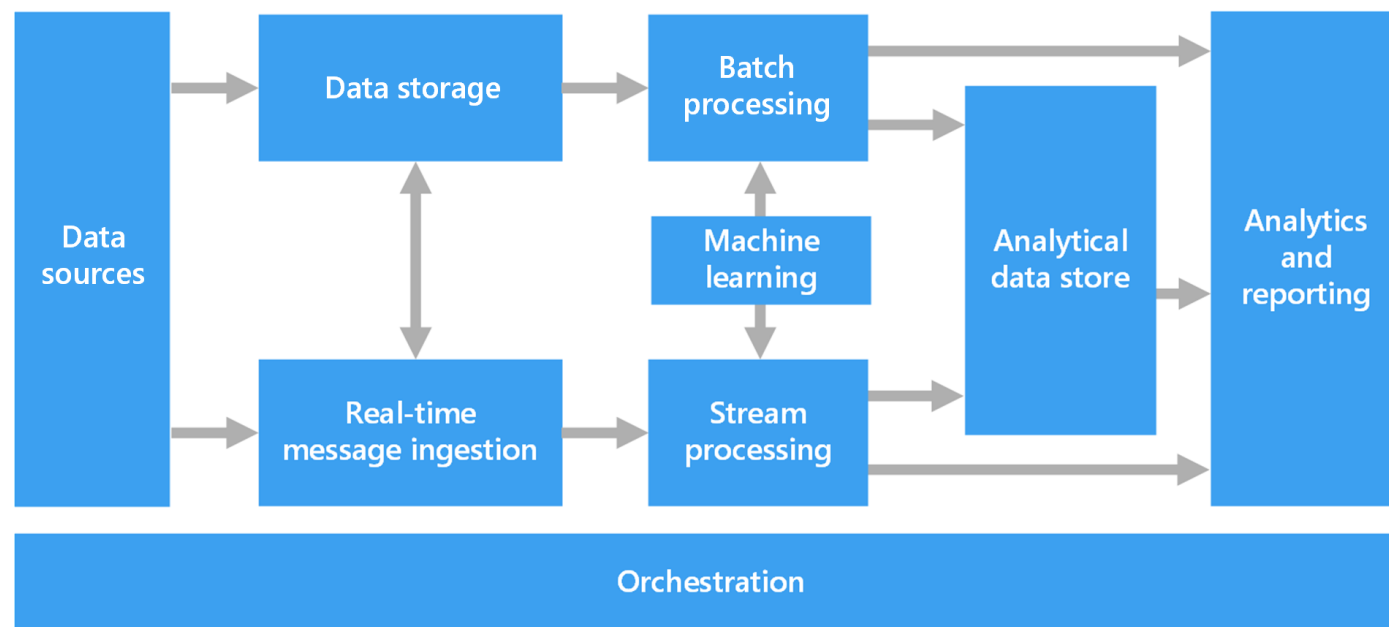
Cloud Tactics:

AWS Placement Groups: Spread and Partition
Anti-affinity on k8s: Pod or Node

Arch Styles: Big Compute



Arch Styles: Big Data



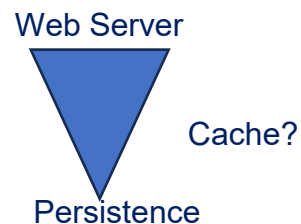
Performance using parallelism.

Storage on DataLake, HDFS (S3) using Partition (AWS S3)

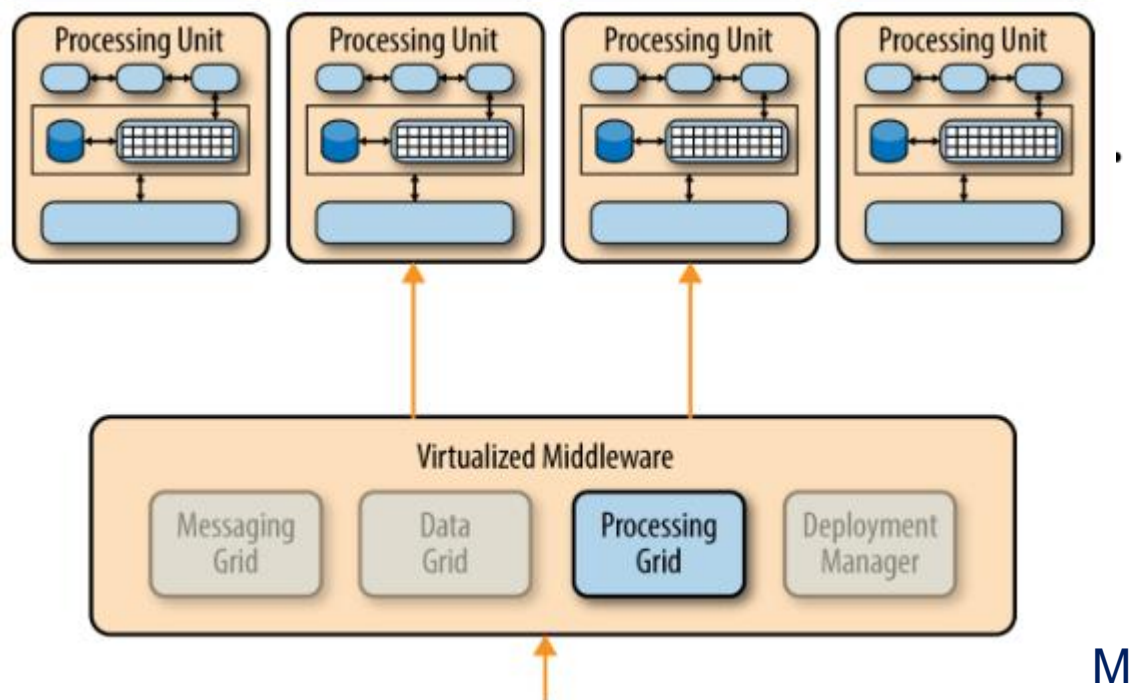
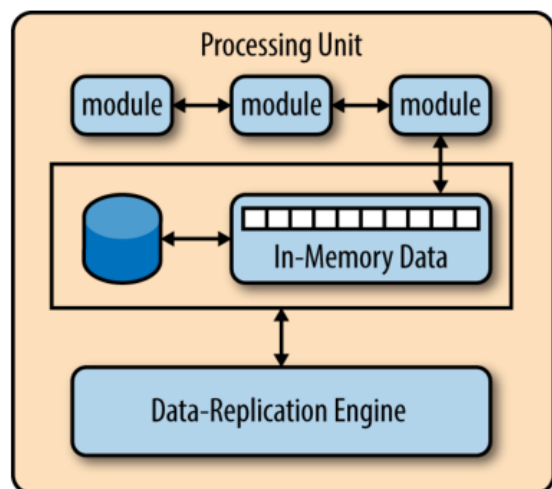
Stream processing on flight using Shards or Partitions (Kafka, AWS Kinesis).

Architectures for it: [Lambda and Kappa](#).

Space-Based Arch



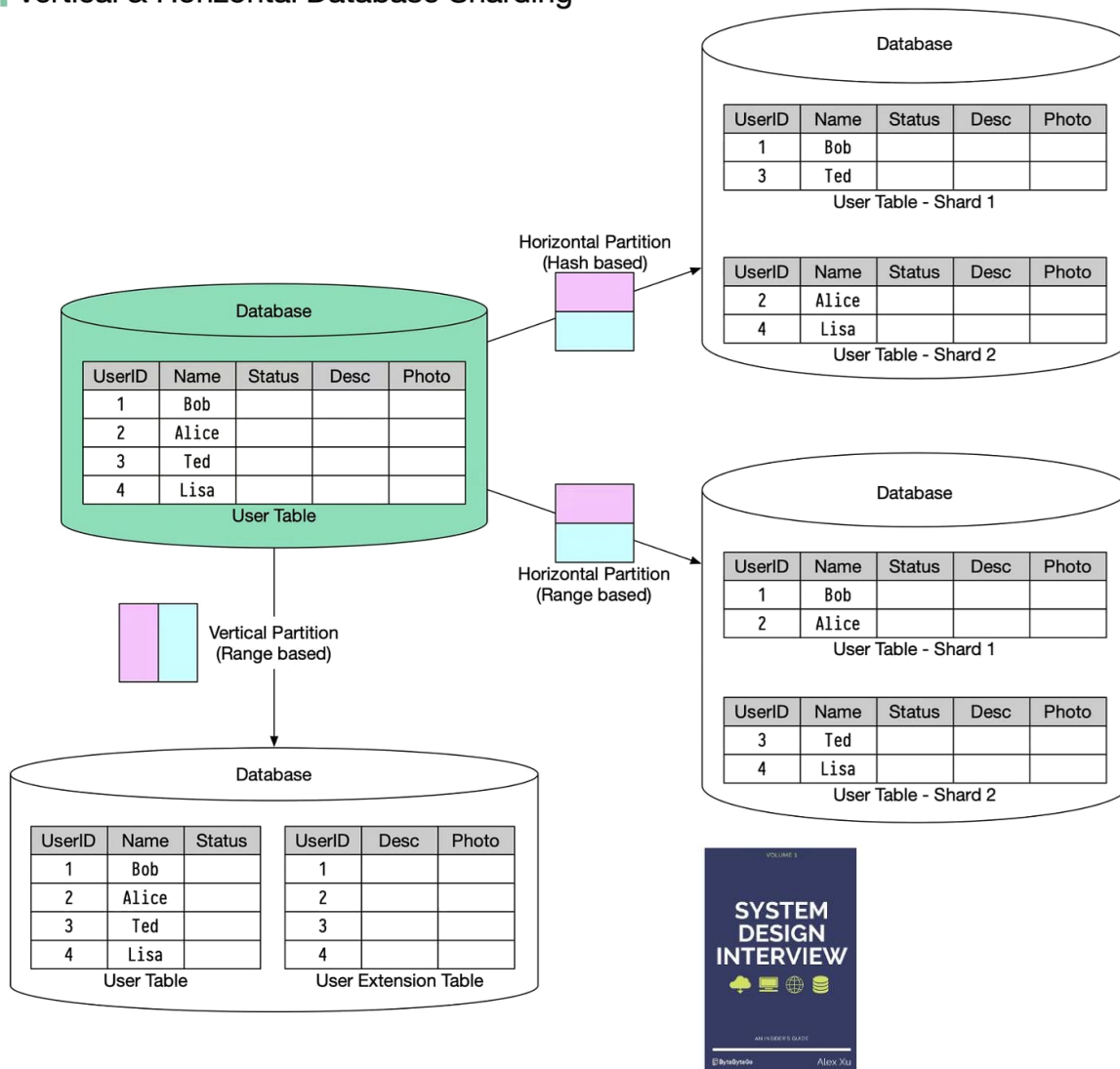
Scenario: Auction Platform (Where is the bottleneck according scalability?).
 Extreme scale, concurrency and performance.
 Solution: Distributed shared memory, replicated in-memory data grids



Messaging Grid: Cmds
 Data Grid: Orch. Data ops
 Processing: >1 PU involved
 More info at FoSA, Ch. 16

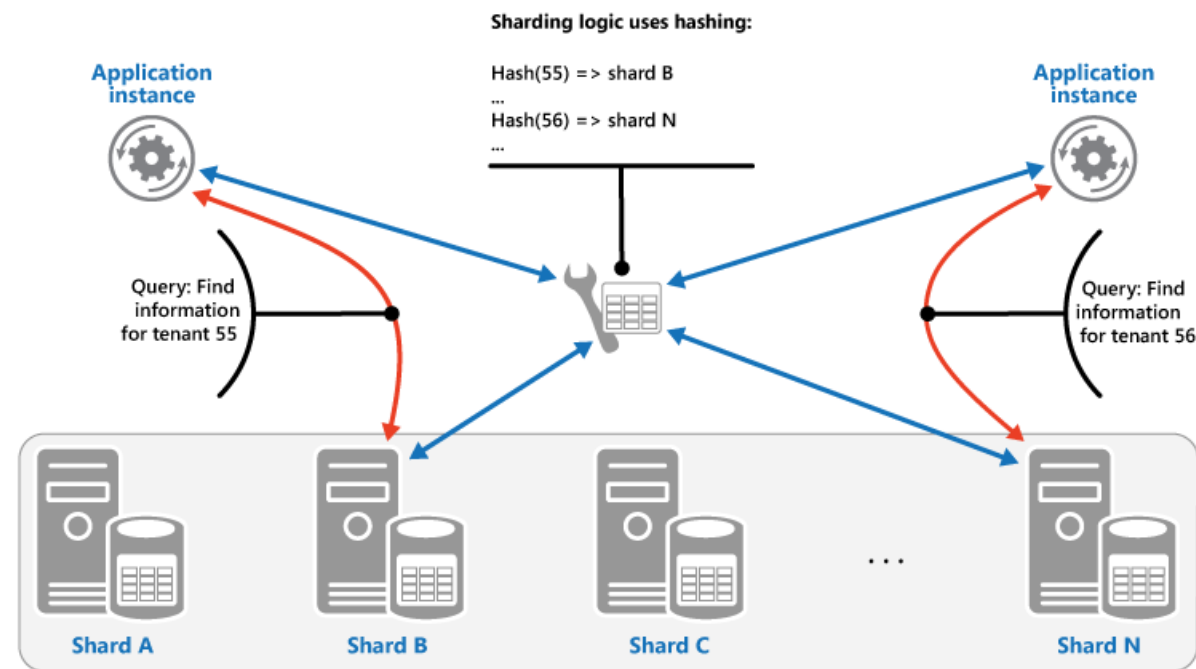
Sharding or horizontal partitions

Vertical & Horizontal Database Sharding



Partition, literal on the same component instead of multiple (sharding).

Lookup (directory), range-based, hashed-based, geo sharding strategy.



Design shard key

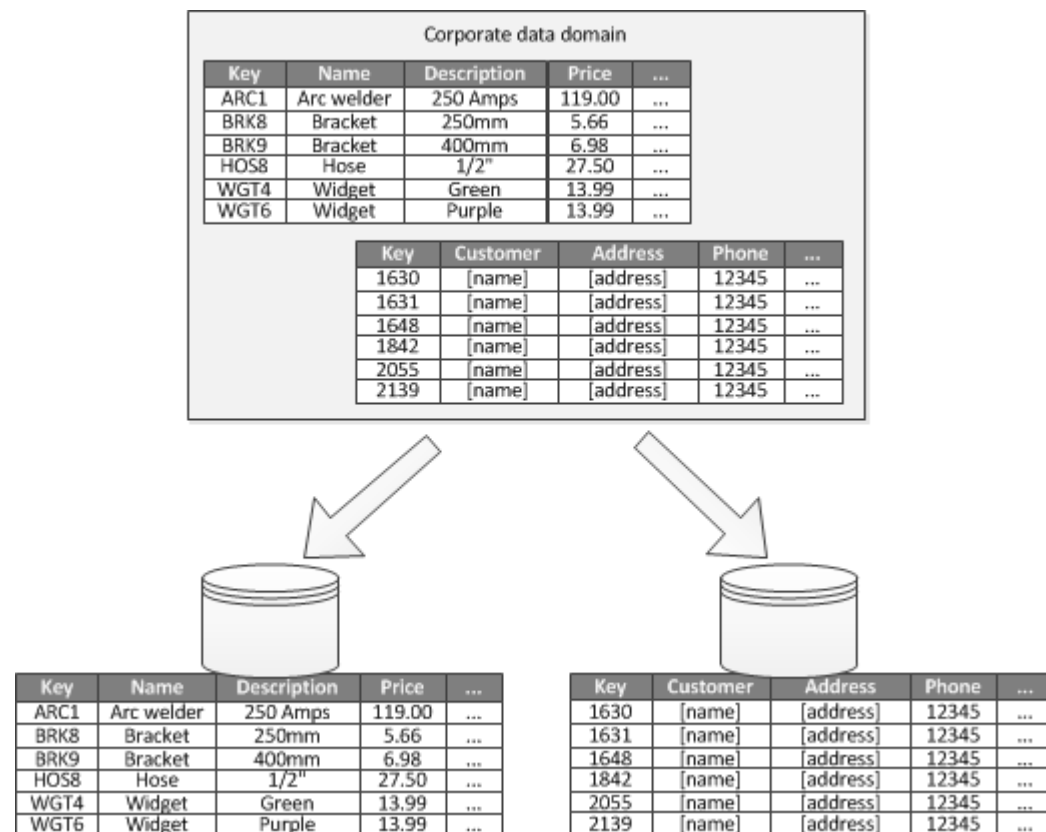
Even data distribution (avoid skew).

Factor	Possible values
Cardinality	Yes/no [max. 2]
Frequency	Age range in a gym [depends on bins, >50]: biz value
Monotonic change	Biz change

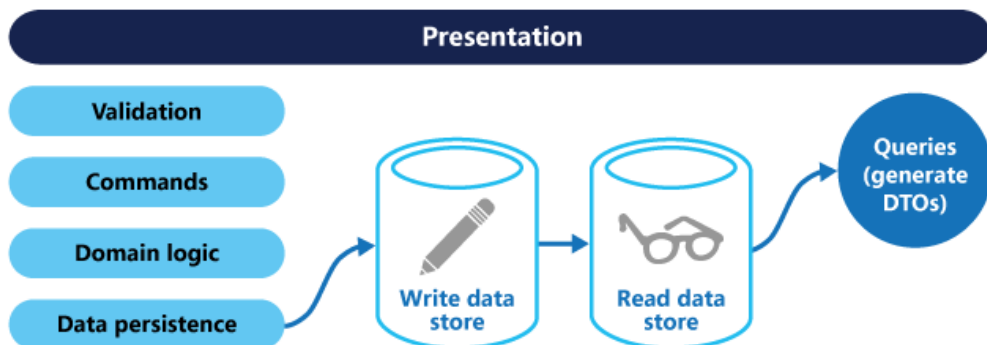
Ops Challenges

- Data hotspots (Choose a shard key)
- Operational complexity
- Infrastructure costs
- Application complexity (Cross-shard JOINS queries)

Other ways to partitioned: Bounded Context. [Link](#)



CQRS Pattern



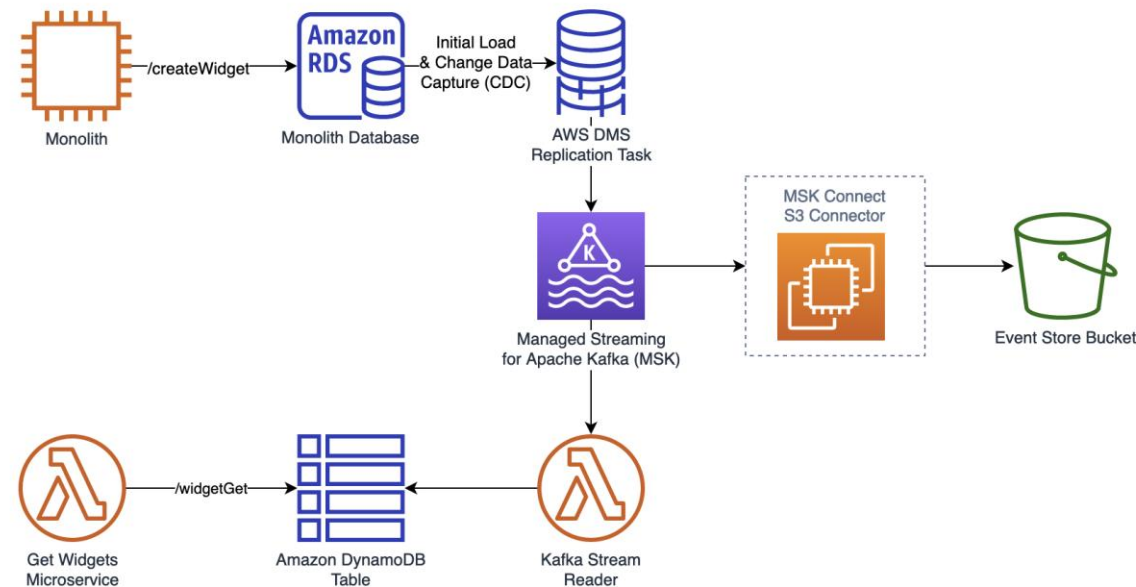
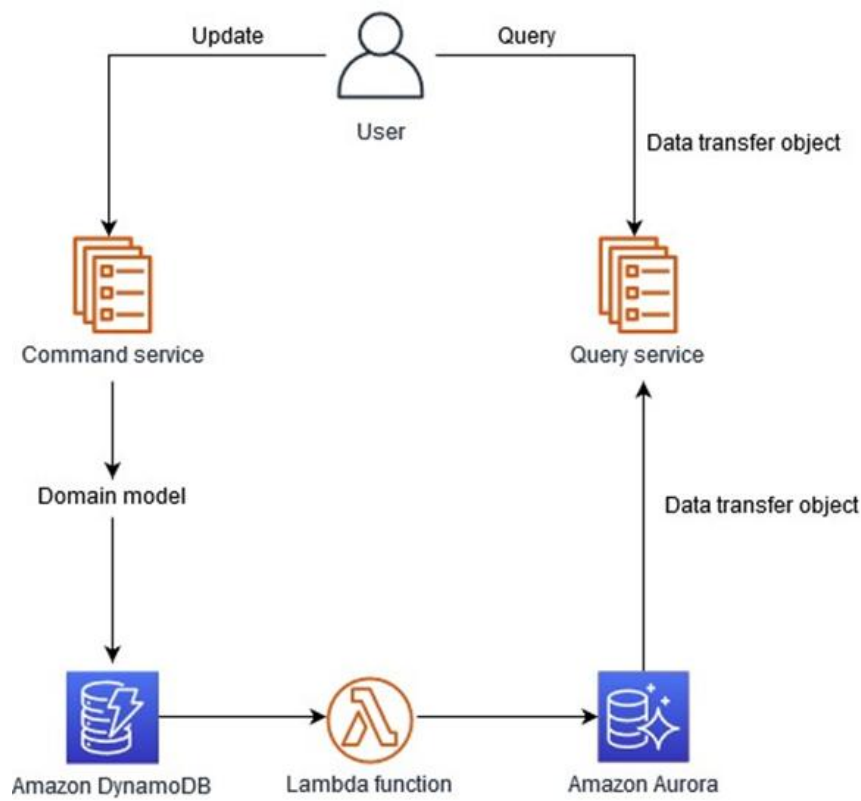
Command Query Responsibility Segregation

Separate needs for W (Cmds) and R (Query)

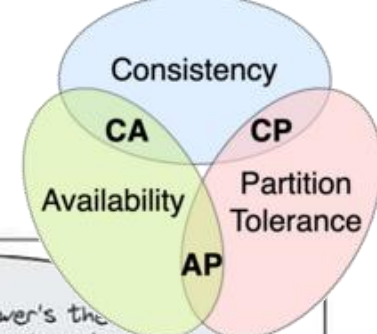
SoC for lower layer

Synch: Eventual Consistency, can use EDA, Transactional Outbox (Tmp store to broker to event publish).

Simple implementation: RDS RR, DynamoDB streams, DMS Replication tasks and MSK.



CAP and PAECL



Partition is a **must** due to the need of Resiliency. Therefore, you have to **survive** choosing CP or AP.

About our Patterns ?

CQRS (AP): Availability → Consistency (Eventual).
 CQRS (L): Latency → Consistency (Eventual).

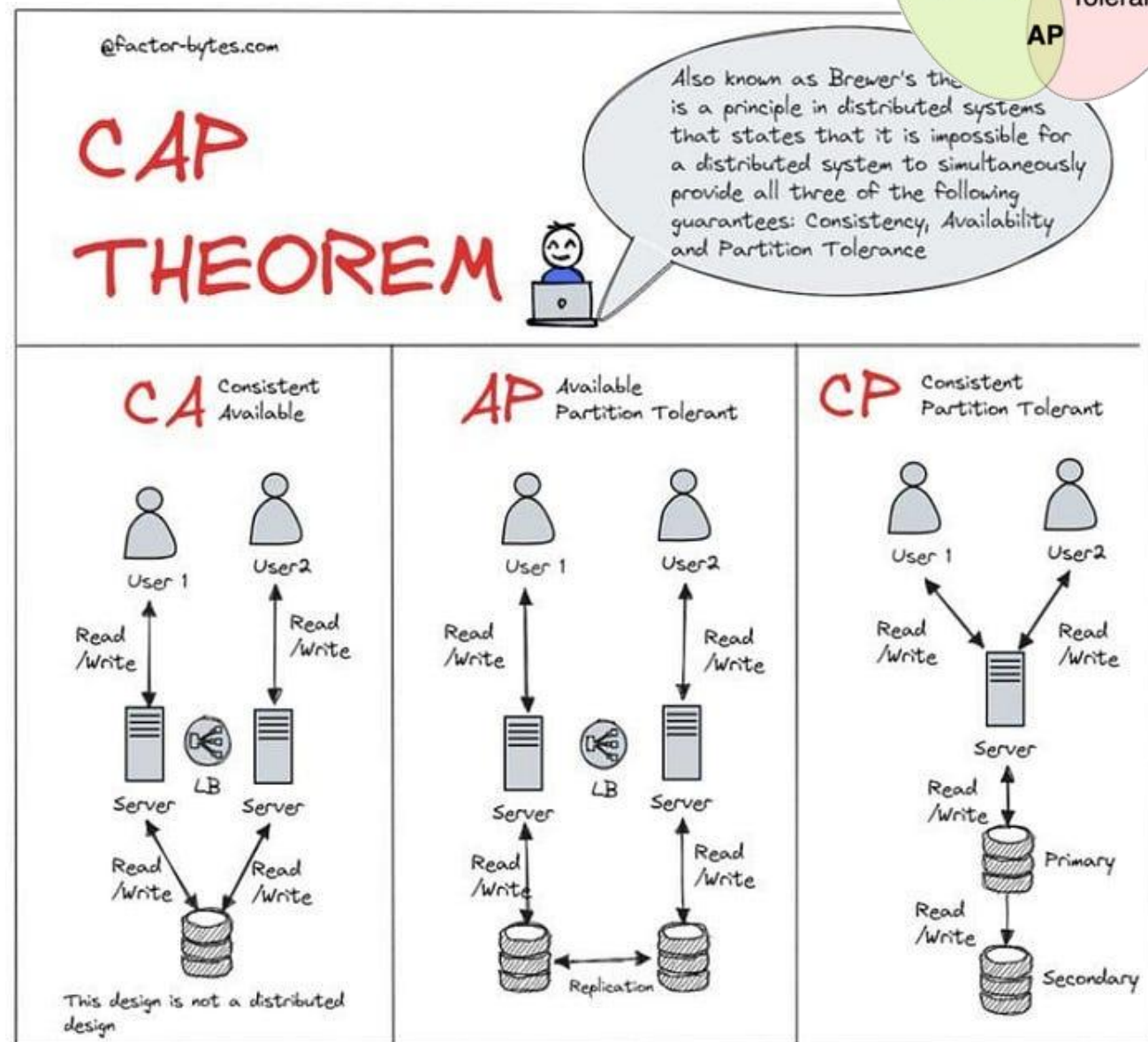
Sharding (CA/P): Depends. AP-→Normal, a node failure enter to inconsistency. Node Redundancy (Leader/Quorum). CP-→2PC (Block).

Sharding (LC): Depends. PA/EL (DynamoDB), PA/EC (MongoDB).

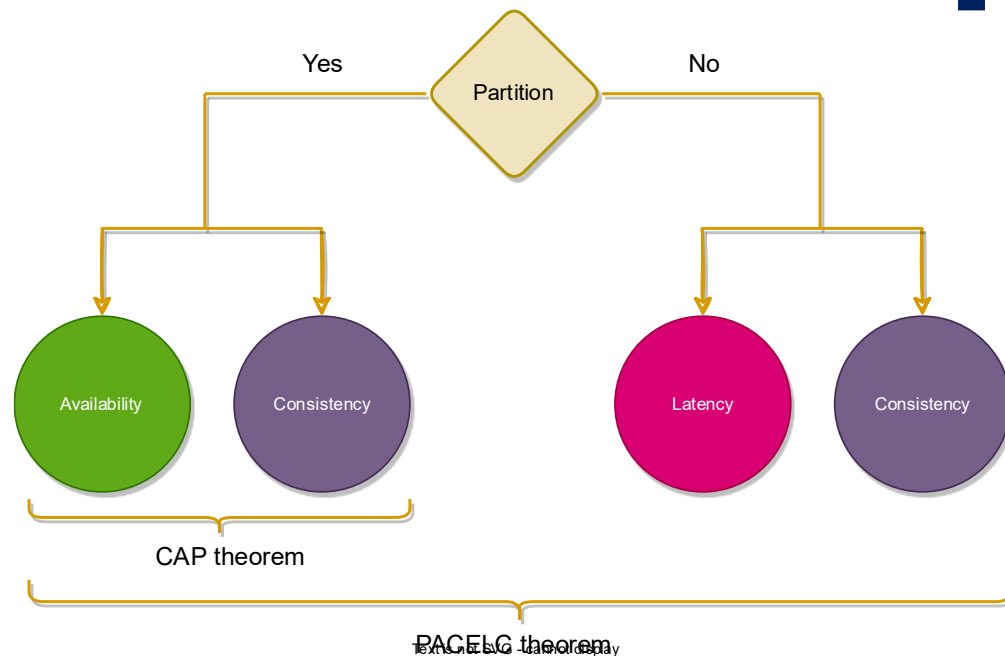
Can you notice on AWS Managed-DBs Features ?

AP is

CP is...



Databases mapped to PACELC



Here the partition is networking partition (failures or degradation).

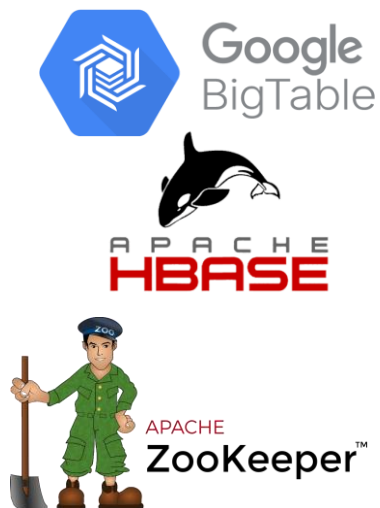
How to jump to PACELC ?

- Routing -> MongoDB
- Cost -> DynamoDB (Capacity Units)
- Primary/Leader Selection
- Multiple replications
- Internal synch

PA/EL



PC/EC



PA/EC



Amazon DynamoDB

Developer Guide

What is Amazon DynamoDB?

▶ Getting started with DynamoDB

Developer Guide

What is Amazon DynamoDB?

Getting started with DynamoDB

How it works

Reads and write

Best Practices Guide for MongoDB

More than 20 tips for getting started

[Documentation](#) > [Amazon DynamoDB](#) > [Developer Guide](#)

Best practices for designing and architecting with DynamoDB

PDF

RSS

Markdown

Focus mode

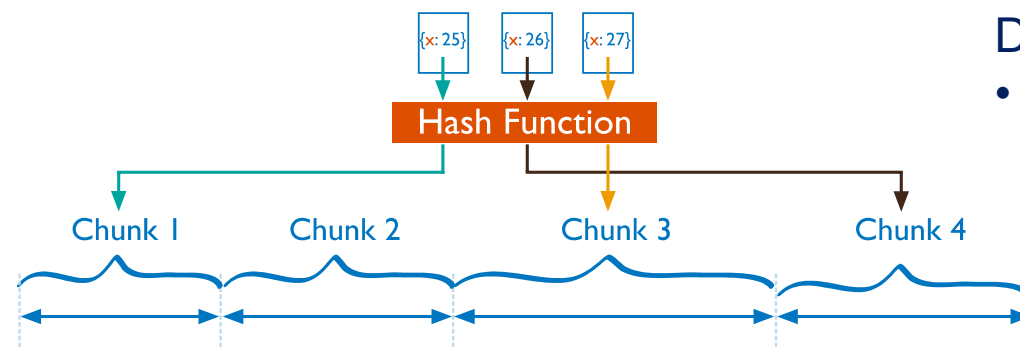
Best practices for designing and using partition keys effectively in DynamoDB

PDF

RSS

Markdown

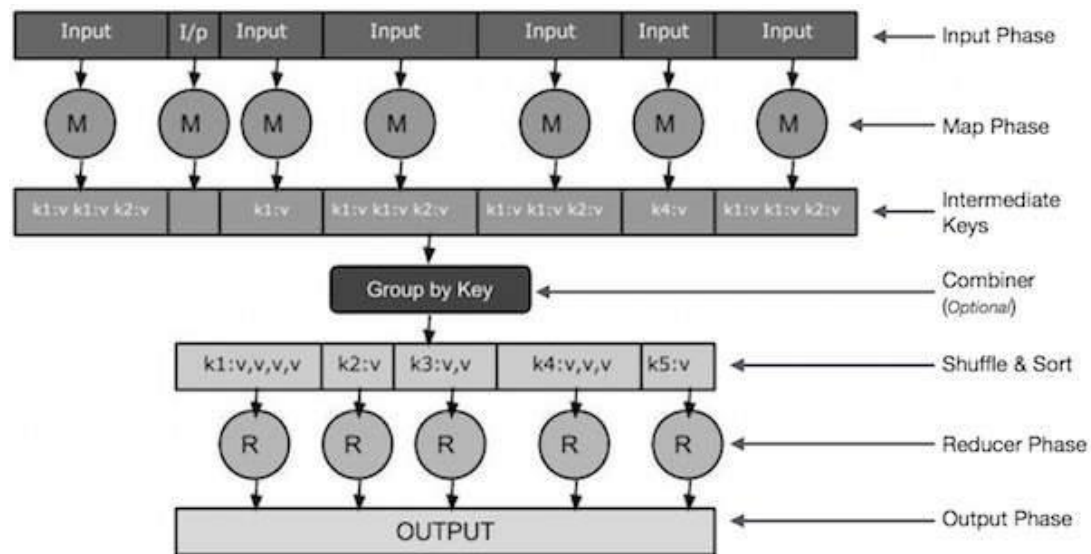
Focus mode



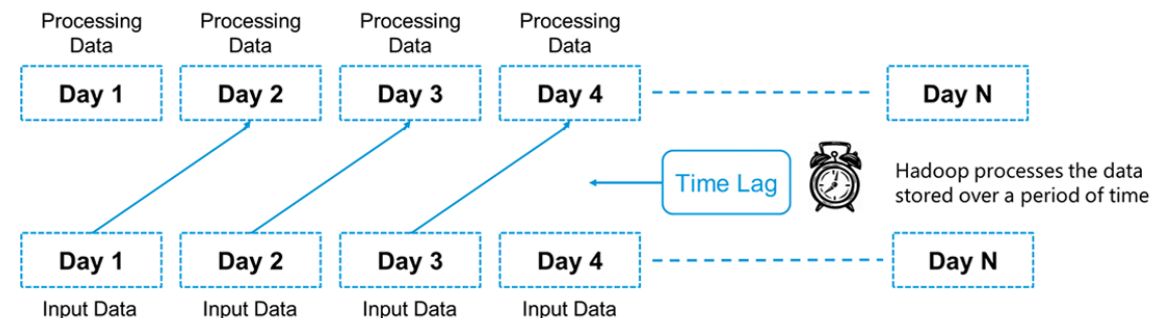
DynamoDB:

- The difference between Scan and Query ?

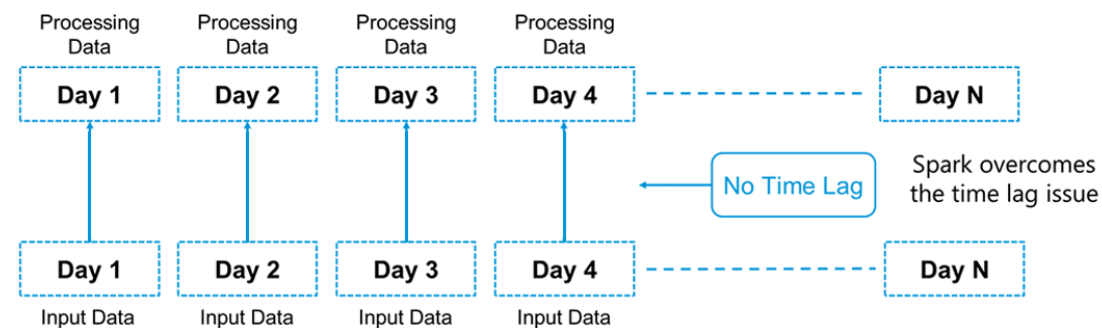
MapReduce and Spark



Processing Data Using MapReduce



Real Time Processing in Spark



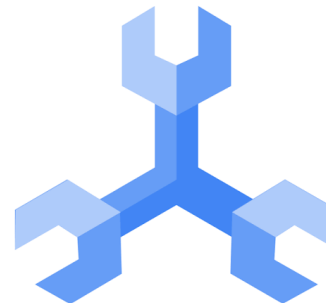
Feature	Hadoop MapReduce	Apache Spark
Intermediate Storage	Disk-Based: Writes the state to HDFS after every <i>map</i> and <i>reduce</i> phase (materialization).	In-Memory: Keeps transformed data in RAM, avoiding constant disk I/O.
Computation Model	Rigid Linear: Strictly limited to successive Map and Reduce phases.	DAG (Directed Acyclic Graph): Compiles operations into an optimized pipeline before execution.
Latency	High: Designed specifically for high-volume batch processing; not suitable for low-latency tasks.	Low: Supports near real-time processing, micro-batching, and interactive queries.
Iterative Algorithms	Inefficient: Each iteration must read from and write to disk (e.g., in Machine Learning).	Highly Efficient: Up to 100x faster by reusing data in memory across multiple steps.

Distributed Relational Databases

And the best of both worlds ???

From CSPs:

- Google Cloud Spanner: Atomic Clock for consistency
- Amazon Aurora
- Azure SQL Database Hyperscale or Cosmos DB



From Open Source world:

- CockroachDB
- YugabyteDB
- TiDB



From PostgreSQL Extensions:

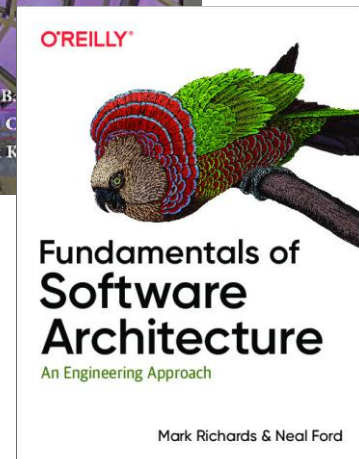
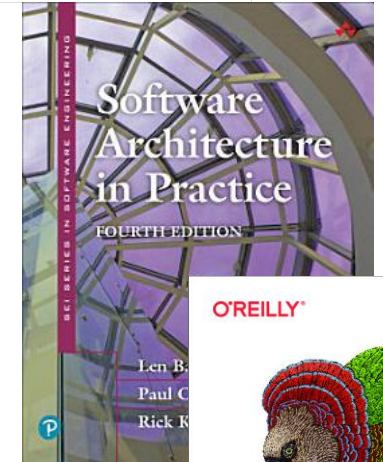
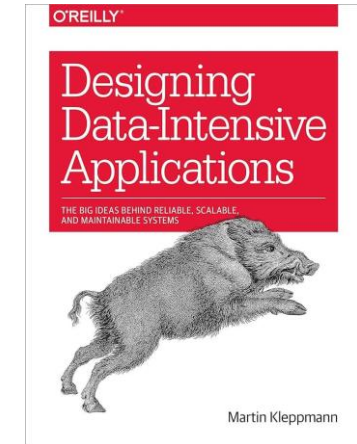
- Citus (by Microsoft)
- ScyllaDB / TimescaleDB



ScyllaDB

References

- [DDiA] Kleppmann, Martin. Designing Data-Intensive Applications (2017), 1st Edition. <https://learning-oreilly-com.ez.unisabana.edu.co/library/view/designing-data-intensive-applications/9781491903063/>
- [AwCQ] Amazon Web Services. (n.d.). *Patterns for enabling data persistence*. AWS Prescriptive Guidance. <https://docs.aws.amazon.com/prescriptive-guidance/latest/modernization-data-persistence/enabling-patterns.html>
- [AzDe] Microsoft. (n.d.). Architecture styles. Azure Architecture Center. <https://learn.microsoft.com/en-us/azure/architecture/patterns/>
- [AwSH] Amazon Web Services. (n.d.). *What is Database Sharding?*. <https://aws.amazon.com/what-is/database-sharding/>
- [FoSA] Richards, M., & Ford, N. (2025). Fundamentals of software architecture (2nd ed.). O'Reilly Media. <https://learning-oreilly-com.ez.unisabana.edu.co/library/view/fundamentals-of-software/9781098175504/>
- [DyBP] Amazon Web Services. (n.d.). Amazon, Best practices for designing and architecting with DynamoDB. AWS Documentation. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/best-practices.html>
- [DyPk] Amazon Web Services. (n.d.). Best practices for designing partition keys. AWS Documentation. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-partition-key-design.html>
- [MoSh] MongoDB. (n.d.). Sharding. MongoDB Manual. <https://www.mongodb.com/docs/manual/sharding/>
- [MoBP] MongoDB. (n.d.). Best practices guide for MongoDB. MongoDB Resources. <https://www.mongodb.com/resources/products/fundamentals/best-practices-guide-for-mongodb>
-





Universidad de
La Sabana

FACULTAD DE INGENIERÍA

CONTACTO

Francisco Moreno

Profesor Catedra – Facultad de Ingeniería

franciscomodi@unisabana.edu.co